

Hands-on shallow-water modelling with Python

December 14, 2025

What does this shallow-water model do?

- 2D shallow-water equations on an **Arakawa C-grid**
- Rectangular basin on an **f-plane** or β -plane
- Options for:
 - Linear / nonlinear momentum equations
 - Rayleigh friction and viscosity
 - Range of forcings (wind, tides, storms, etc.)
- Designed to be:
 - **Small and readable**: good for learning and experimenting
 - **Modular**: separate grid, parameters, forcings, numerics, plots
- Comes with:
 - A **public Python API** for scripts and notebooks
 - Example **Jupyter notebooks** for classical ocean-dynamics problems

Tour of the repository

Top level

- `README.md` – quick start, installation, examples
- `notebooks/` – study cases (wind gyre, Rossby waves, gravity waves, tides, storms...)
- `pyproject.toml`, `setup.cfg` – packaging info

Main code (Python package)

`src/shallowwater/`

- `__init__.py` – exposes the public API
- `grid.py` – grid creation, metric factors (C-grid)
- `params.py` – `ModelParams` and physical/numerical parameters
- `initial.py` – initial conditions (Gaussian bumps, balanced states, ...)
- `forcing.py` – wind stress, tidal forcing, storms, ...
- `tendencies.py`, `time_stepping.py` – core numerical scheme
- `visualize.py` – helper functions for plots and animations

Mental map

User-facing: `__init__.py`, `notebooks/`, `visualize.py`

Under the hood: `grid.py`, `tendencies.py`, ...

Public API cheat sheet

Model configuration

- `ModelParams(...)`
Physical: `H`, `g`, `rho`, `f0`, `beta`, `y0`
Damping/viscosity: `r`, `Ah`, ...
Switches: `linear=True/False`, toggle advection, ...
- `make_grid(Nx, Ny, Lx, Ly)` builds the computational grid

Initial conditions and forcing

- `setup_initial_state(grid, params, mode=..., ...)`
e.g. `mode="gaussian_bump"`, `mode="balanced_jet"`
- Forcing functions like `zero_forcing`, `wind_gyre_forcing`, ...
- You can define your **own forcing function** with the same signature

Time stepping and visualisation

- `compute_dt_cfl(grid, params, cfl=0.5)` suggests a stable time step
- `run_model(tmax, dt, grid, params, forcing_fn, ic_fn, ...)`
integrates the equations
- Visualisation helpers: `animate_eta(...)`, `plot_forcings(...)`,
Hovmöller/spectral plots

Where to find the shallowwater package

- The shallowwater code is hosted on GitHub:
`https://github.com/fabien-roquet/shallowwater.git`
- We will use the tagged version:
tag: v0.1.1
- **What is GitHub?**
 - Online platform to store and share code.
 - Uses git for version control (history of changes).
 - Makes it easy to collaborate (issues, pull requests).
- The package is also on the Python Package Index (PyPI) and can be installed with:
`pip install shallowwater`
- In this lab you will:
 - Download the GitHub repository locally (to see the code, docs, examples).
 - Install the library via `pip install shallowwater` (to use it in Python).

Downloading the repository with git

- If you have git installed (Linux, macOS, or Windows):
 - Open a terminal (or a terminal inside JupyterLab).
 - Go to a folder where you want to store the code, e.g.:
`cd ~/projects`
 - Clone the repository:
`git clone https://github.com/fabien-roquet/shallowwater.git`
`cd shallowwater`
 - Switch to the tagged version v0.1.1:
`git checkout v0.1.1`
- You now have a local copy of:
 - Python source code in `src/shallowwater/`
 - Example notebooks in `notebooks/`
 - Documentation and configuration files

Downloading the repository without git

- If you do not have git (common on Windows), you can use a ZIP file instead:
 - Open the GitHub page in your web browser.
 - Click "**Code**" → "**Download ZIP**".
 - Save the ZIP file to your computer.
 - Unzip it to a folder, for example:
Documents/shallowwater/
- After unzipping:
 - You can see the same contents as in the git clone (code, notebooks, etc.).
 - You can point JupyterLab's file browser to that folder to open files.
- Optional: check the version information (in the README or release notes) to confirm it corresponds to version 0.1.1.

Installing shallowwater from PyPI

- Once Python and pip are available, install the package with:
`pip install shallowwater`
- This fetches the released version from PyPI and makes it available as a normal Python library.
- You can quickly verify that the installation works:
`python -c "import shallowwater"`
- After installation, you can `import shallowwater` in any Python script or notebook, regardless of where the GitHub repo sits.
- No special virtual environment is strictly required here:
 - Only `numpy` is needed.
 - `numba` (if installed) is used optionally for speed.

Using the `lab_tsunami.ipynb` notebook

- The lab material is provided separately on Canvas as:
`lab_tsunami.ipynb`
- Steps:
 - Download `lab_tsunami.ipynb` from Canvas.
 - Save it in a convenient folder, for example inside your local `shallowwater` directory.
- Start JupyterLab (from your usual method) and:
 - Use the file browser to navigate to the folder where `lab_tsunami.ipynb` is stored.
 - Open the notebook by clicking on it.
- The notebook will:
 - `import shallowwater` (installed via `pip install shallowwater`).
 - Guide you through running a tsunami-like simulation.
 - Ask you to experiment with key parameters:
 - water depth H
 - initial Gaussian disturbance